



KUBERNETES

Review of Containerization & Kubernetes

Containers provide packaging and portability, while Kubernetes enables orchestration at scale—together forming the foundation of modern AI deployment pipelines.

Why Containers for AI?

Standardized Environments

Eliminates "works on my machine" problems by ensuring consistent execution across development, testing, and production.

Complete Packaging

Bundles code, dependencies, and libraries into a single deployable unit that runs identically everywhere.

Lightweight Virtualization

Significantly faster startup and lower overhead compared to traditional VMs, enabling efficient resource utilization.

Reproducibility

Critical for AI experiments—ensures results can be validated and replicated by capturing the exact execution environment.



CONTAINER CONCEPT ICONS



Core Container Concepts

Image

A read-only snapshot containing application code, runtime, libraries, and dependencies—everything needed to run your AI application.

Container

A running instance of an image that executes in an isolated environment with controlled access to host resources.

Registry

A repository for storing and distributing container images (DockerHub, Amazon ECR, Google GCR, Harbor).

Dockerfile

A text file containing instructions for building an image—specifies base image, dependencies, environment variables, and startup commands.

Containers in AI Workloads

Modern AI infrastructure relies on containers to solve critical deployment challenges:

Model Serving

Package ML models with inference servers (FastAPI, Flask, TorchServe, Triton) for consistent deployment.

Versioning

Capture model artifacts, code, and dependencies in immutable images for reproducibility and rollbacks.

Distributed Computing

Distribute training and inference workloads across clusters with identical environments.

Framework Flexibility

Enable multiple ML frameworks (PyTorch, TensorFlow, ONNX) to coexist in the same infrastructure.



Kubernetes Overview

Kubernetes is a production-grade container orchestration system that manages deployments, provides automatic scaling, and enables self-healing infrastructure.

Core Objects

1

Pod

Smallest deployable unit containing one or more containers that share storage and network.

2

Deployment

Defines desired state, scaling rules, and update strategies for a set of pods.

3

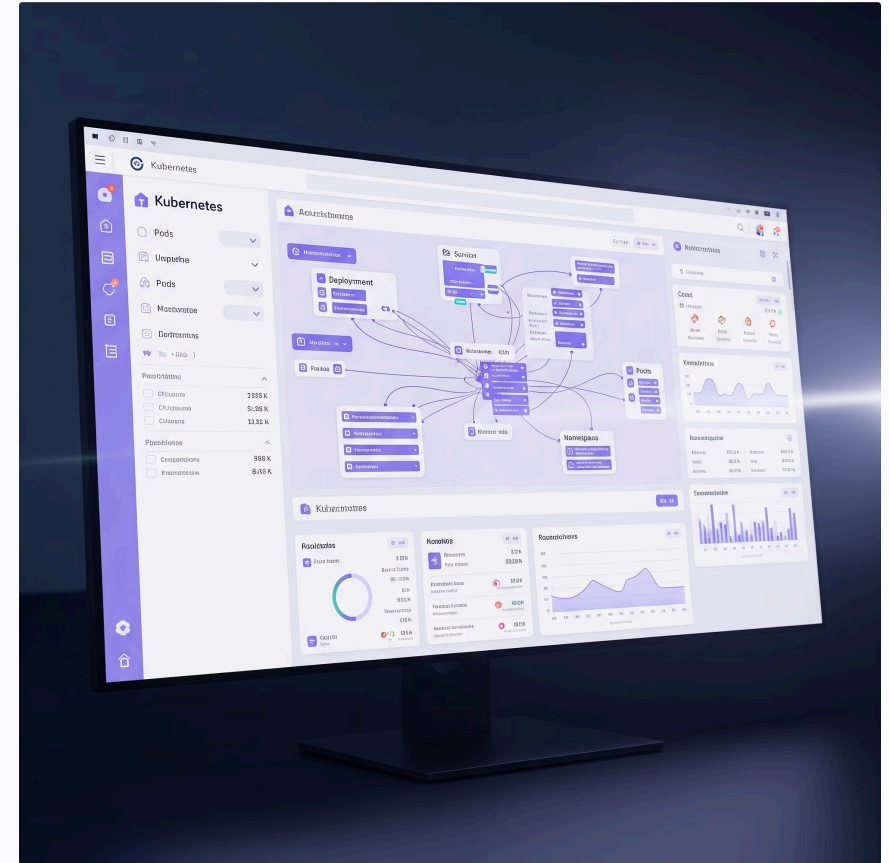
Service

Exposes pods to network traffic and provides stable endpoints as pods come and go.

4

Namespace

Provides logical isolation for teams, projects, or environments within a cluster.



Kubernetes for AI Infrastructure



Kubernetes provides the foundation for scalable, resilient AI infrastructure:

- **Training Orchestration:** Deploy training jobs on GPU-enabled pods with resource guarantees
- **Autoscaling:** Scale inference services automatically based on traffic patterns
- **Multi-tenancy:** Isolate teams and workloads with namespaces and resource quotas
- **MLOps Integration:** Connect with specialized tools like Kubeflow, MLflow, and Argo Workflows
- **Resource Efficiency:** Pack workloads efficiently to maximize hardware utilization

GPU Support in Kubernetes

GPU acceleration is essential for modern AI workloads:

NVIDIA Device Plugin

Enables GPU scheduling and allocation in Kubernetes clusters, exposing hardware capabilities to pods.

Resource Requests

Pods can request specific GPU resources:

```
resources.limits.nvidia.com/gpu: 1
```

Multi-GPU Workloads

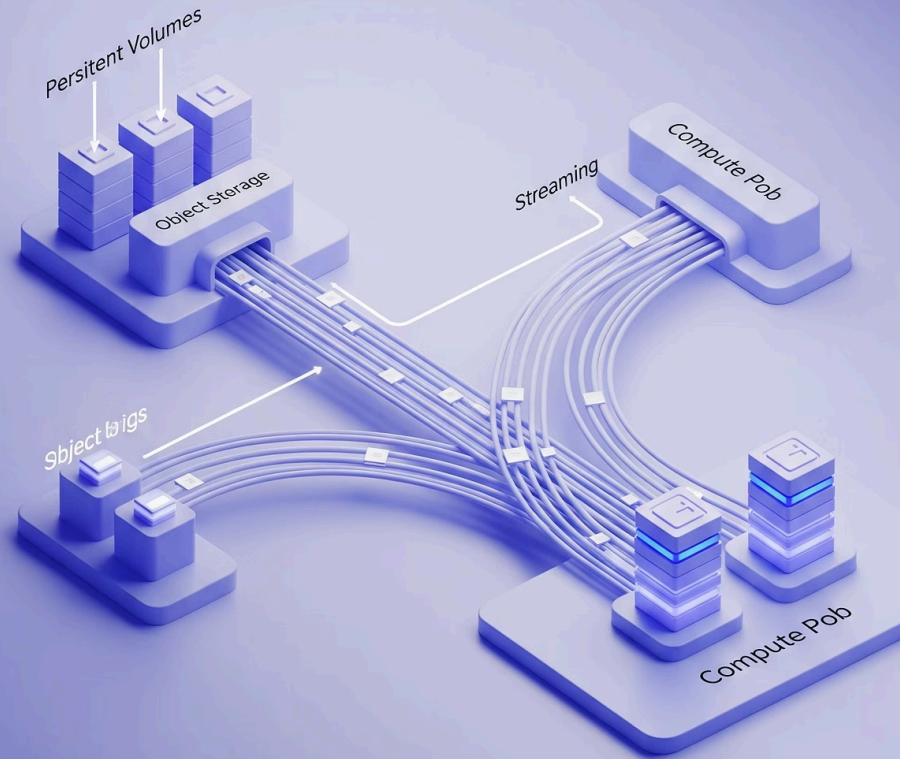
Distributed training across multiple GPUs using frameworks like Horovod, DeepSpeed, or PyTorch DDP.

GPU Monitoring

Track utilization, memory usage, and temperature using DCGM and Prometheus.



Kubernetes Storage & Data Pipelines



Persistent Volumes

Kubernetes PVs provide durable storage for datasets and model artifacts that persists beyond pod lifecycles.



Object Storage Integration

CSI drivers connect pods to S3, GCS, and Azure Blob for scalable, cost-effective storage of training data and models.



Streaming Data Pipelines

Kafka connectors and Kubernetes-native event processing enable real-time data flows to training and inference pods.



Parallel Processing

Orchestrate complex data preparation and training pipelines with parallelism for maximum throughput.

Monitoring & Scaling in Kubernetes



Scalable AI Infrastructure

- **Horizontal Pod Autoscaler (HPA)** automatically scales inference deployments based on CPU, memory, or custom metrics
- **Custom Metrics Adapters** enable scaling on AI-specific metrics like GPU utilization, inference latency, or queue depth
- **Prometheus + Grafana** provide comprehensive monitoring of cluster health, resource usage, and application performance
- **Canary Deployments** allow safe testing of new model versions by routing a percentage of traffic to updated pods
- **Node Autoscaling** adds or removes worker nodes based on cluster-wide resource demands

Case Study: OpenAI on Kubernetes

Challenges

- Massive inference demand with unpredictable spikes
- GPU-intensive workloads requiring specialized hardware
- Global user base needing low-latency responses
- High availability requirements for production APIs

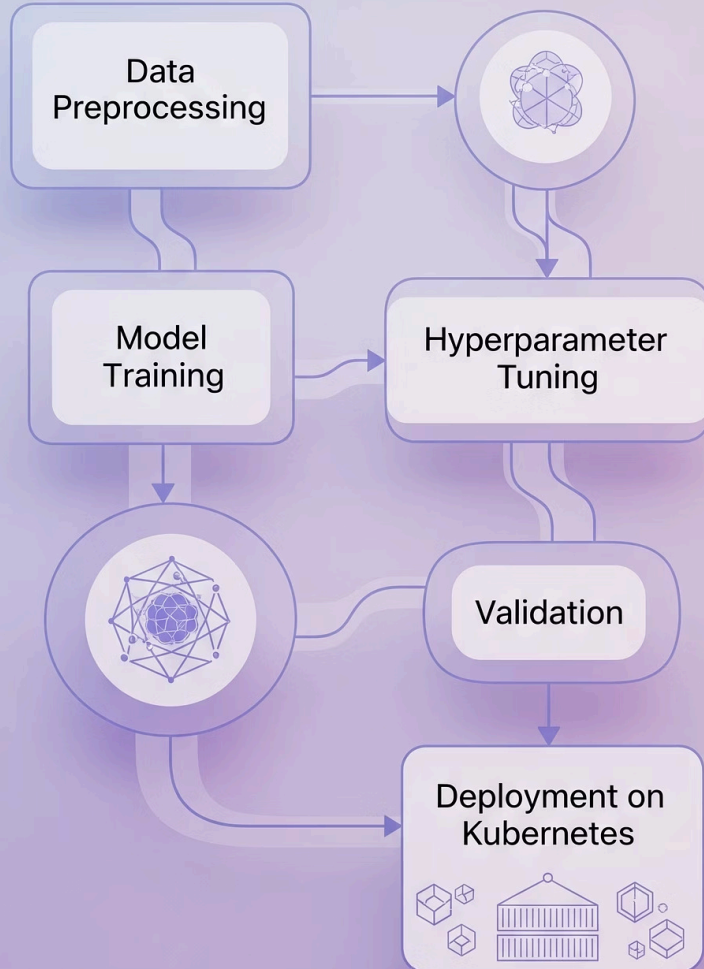
Kubernetes Solution

- Containerized inference APIs deployed in Kubernetes
- GPU pods dynamically scaling based on demand
- Multi-region deployment for latency optimization
- Orchestration for global reliability and fault tolerance



Result: Scalable, resilient infrastructure serving billions of API requests globally

Kubeflow



Case Study: Kubeflow Pipelines

Kubeflow extends Kubernetes with ML-specific orchestration capabilities, creating an end-to-end MLOps platform.

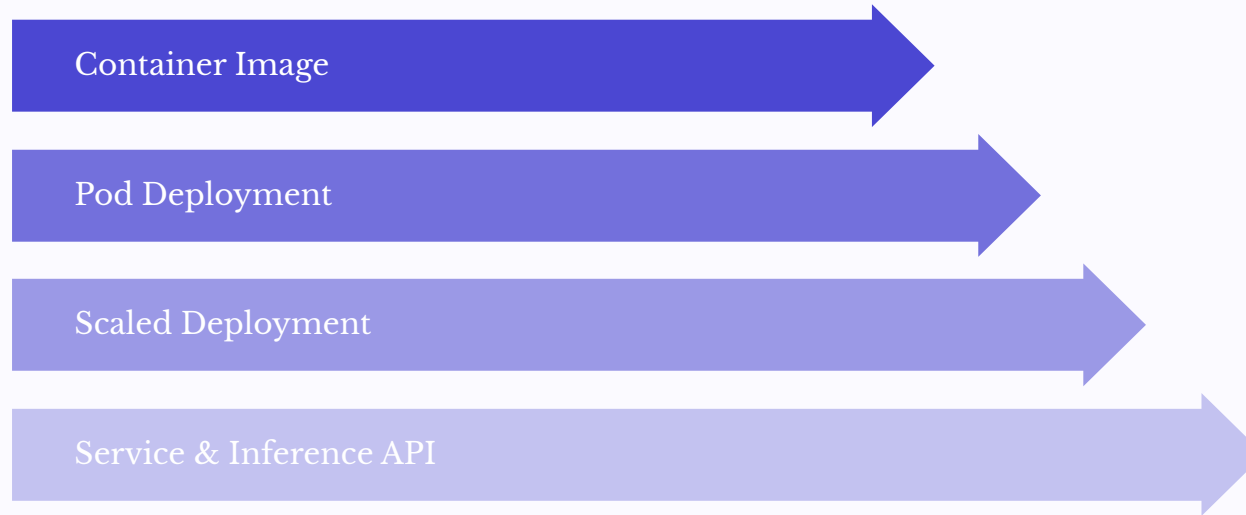
Key Benefits

- Automates entire ML lifecycle from preprocessing to deployment
- Enables CI/CD workflows specific to ML model development
- Provides reproducible, parameterized pipeline definitions
- Integrates with experiment tracking and model registry

Production Impact

- Modular, reusable components accelerate development
- Infrastructure becomes reproducible and version-controlled
- Model governance and compliance are simplified
- Deployment becomes consistent and automated

Containers to Inference: The Journey



"Containers package, Kubernetes orchestrates."

This architecture provides the scalability, reliability, and operational efficiency required for production AI systems. Each component plays a critical role in translating model code into a robust service.

Key Lessons

1

Standardization

Containers create consistent AI deployment environments across development, testing, and production—eliminating "works on my machine" problems.

2

Orchestration

Kubernetes enables elasticity, self-healing, and automated scaling—critical capabilities for production AI systems with variable workloads.

3

Acceleration

GPU scheduling in Kubernetes makes specialized hardware available for training and inference, maximizing performance while managing costs.

4

Integration

MLOps tools like Kubeflow and Argo extend Kubernetes with ML-specific workflows, creating end-to-end platforms for the AI lifecycle.

Summary

Containers

Building blocks for portable
AI applications with
consistent environments

Capstone

Essential knowledge for
your upcoming capstone
project deployment



Kubernetes

Orchestrator providing
scaling, resilience, and
resource management

Infrastructure

Foundation of modern AI
operations enabling reliable
production deployments



Together, containers and Kubernetes
form the backbone of scalable,
reproducible AI infrastructure—
transforming how we build and deploy
intelligent systems.

Discussion Prompt

If you were deploying an LLM service, would you prioritize multi-cluster Kubernetes (global scale) or single high-availability cluster (simplicity) — and why?

Multi-Cluster Considerations

- Global reach with lower latency
- Regional data sovereignty compliance
- Higher availability across cloud outages
- Added operational complexity
- More challenging networking setup

Single HA Cluster Considerations

- Simpler operations and monitoring
- Lower management overhead
- Easier networking and security
- Potential single region latency issues
- Limited fault isolation boundaries

